Ain Shams University
Faculty of Engineering
Computer Engineering and Software Systems Program

CSE 335: Operating Systems – Summer2026

TERM PROJECT REPORT

MINIX 3.3.0

Implementation and Native Test Report

Scheduling • Hierarchical Paging • Extent-Aware MFS Allocation

TEAM MEMBERS

Name	Student ID
Jana Ahmed Saieed	24P0410
Mohamed Ehab Abdelbary Ibrahem	2300570
Abdelrahman Ashour Hassan	2101736
Mostafa Hamdy Mohamed Elzoghby	2300672

1. Project summary

The project starts from the official MINIX 3.3.0 release and implements all three requirements as reproducible experiments. Requirement 1 compares four CPU-scheduling policies using real child processes. Requirement 2 models configurable hierarchical page tables and compares FIFO with LRU replacement. Requirement 3 modifies the real MFS zone-allocation path to prefer contiguous free runs, then tests file and directory operations on a disposable MFS image.

Requirement	Implementation	Evidence
1. Scheduling	RR, SJF, priority, and MLFQ dispatcher with real workers	Known-workload PASS; 80 data rows
2. Paging	Sparse configurable hierarchy with FIFO/LRU frames	Known reference-string PASS; 72 data rows
3. MFS extents	Exact-origin bitmap allocation and free-run preference	I/O PASS; 18 rows; zero data errors

Scope and interpretation

The scheduling command controls logical slices but does not replace the production kernel scheduler. The paging command is a native MINIX simulator because x86 hardware page geometry cannot be changed from a configuration file. The MFS work is a real server modification: it changes allocation preference without changing the on-disk format. These boundaries keep the claims accurate.

2. How the project was completed

Stage	What was done
1. Clean baseline	The untouched 3.3.0 release was recorded so every modification is reviewable with git diff.
2. Implement	Three commands, parsers, matrix scripts, deterministic tests, and a focused MFS allocator change were added.
3. Transfer	The source was copied to /usr/src in the MINIX VM for the native compiler and headers.
4. Build	The commands and MFS server were compiled and installed from the submitted source.
5. Correctness gate	Known workloads ran first; a failed assertion or data mismatch would stop collection.
6. Experiment	Scripts varied one main parameter and wrote raw CSV. Extent tests remounted a disposable MFS image.
7. Preserve	CSV matrices, checksums, validation log, and console screenshots were copied into the deliverables.

Compilation and execution inside MINIX were the authoritative gate. Windows-side editing alone was never treated as proof, because only the VM has the correct compiler, headers, services, and runtime behavior.

3. Requirement 1 - scheduling

Implementation

The parent parses the workload, creates one child per job, and uses pipes to grant logical CPU slices. Each child performs deterministic integer work and acknowledges the slice. The parent selects only arrived jobs, records first start and completion, and calculates turnaround, waiting, response, makespan, and dispatch count. RR uses a rotating cursor, SJF selects the smallest remaining burst, priority selects the smallest numeric priority, and MLFQ uses increasing quanta with demotion.

Requirement 1 — scheduler dispatch loop
minix/commands/schedexperiment/scheduler.c

```
314     while (completed < cfg->job_count) {
315         unsigned int slice;
316         struct child_command command;
317         struct child_reply reply;
318         int status;
319
320         if (algorithm == SCHED_RR)
321             selected = select_rr(states, cfg->job_count, now, &cursor);
322         else if (algorithm == SCHED_SJF)
323             selected = select_sjf(states, cfg->job_count, now);
324         else if (algorithm == SCHED_PRIORITY)
325             selected = select_priority(states, cfg->job_count, now);
326         else
327             selected = select_mlfq(states, cfg->job_count, now);
328         if (selected < 0) {
329             selected = next_arrival(states, cfg->job_count, now);
330             if (selected < 0) {
331                 snprintf(error, error_size, "scheduler reached invalid idle state");
332                 close_children(states, cfg->job_count, 1);
333                 return -1;
334             }
335             now = (unsigned int)selected;
336             continue;
337         }
338         if (!states[selected].started) {
339             states[selected].started = 1;
340             states[selected].start_ms = now;
341         }
342         if (algorithm == SCHED_SJF || algorithm == SCHED_PRIORITY)
343             slice = states[selected].remaining_ms;
344         else if (algorithm == SCHED_RR)
345             slice = cfg->quantum_ms;
346         else
347             slice = cfg->mlfq_quantum[states[selected].queue_level];
348         if (slice > states[selected].remaining_ms)
349             slice = states[selected].remaining_ms;
```

Native result

Algorithm	Avg turnaround	Avg waiting	Avg response	Makespan	Dispatches
RR	185 ms	133 ms	17 ms	260 ms	14
SJF	137 ms	85 ms	85 ms	260 ms	5
Priority	155 ms	103 ms	103 ms	260 ms	5
MLFQ	191 ms	139 ms	5 ms	260 ms	15

SJF produced the lowest average waiting and turnaround for this workload. MLFQ produced the quickest first response but required the most dispatches. Makespan is equal because every policy performs the same 260 logical milliseconds of work.

4. Requirement 2 - hierarchical paging

Implementation

The configuration specifies address width, page size, hierarchy levels and index widths, frame count, replacement policy, and deterministic trace parameters. Index bits plus page-offset bits must equal the address width. The sparse hierarchy creates only paths reached by the trace, allowing nodes, entries, and allocated bytes to be measured.

Every byte address is converted to a virtual page. A present page is a hit. On a fault, an unused frame is selected first; otherwise FIFO chooses the smallest load timestamp and LRU chooses the smallest last-access timestamp. First fills count as faults but not replacements.

Requirement 2 — FIFO/LRU frame selection
minix/commands/vmexperiment/simulator.c

```
267 static unsigned int select_frame(struct simulator *sim,
268     enum replacement_policy policy)
269 {
270     unsigned int frame;
271     unsigned int selected;
272     uint64_t selected_time;
273 
274     for (frame = 0; frame < sim->cfg->frames; frame++) {
275         if (sim->frames[frame] == NULL)
276             return frame;
277     }
278     selected = 0;
279     selected_time = policy == POLICY_FIFO ?
280         sim->frames[0]->loaded_at : sim->frames[0]->last_access;
281     for (frame = 1; frame < sim->cfg->frames; frame++) {
282         uint64_t candidate_time;
283 
284         candidate_time = policy == POLICY_FIFO ?
285             sim->frames[frame]->loaded_at :
286             sim->frames[frame]->last_access;
287         if (candidate_time < selected_time) {
288             selected = frame;
289             selected_time = candidate_time;
290         }
291     }
292     return selected;
```

Native result

Page size	FIFO faults	LRU faults	Observed result
1,024 B	6,937	6,791	LRU lower by 146
2,048 B	4,735	4,047	LRU lower by 688
4,096 B	2,666	1,655	LRU lower by 1,011
8,192 B	1,378	1,064	LRU lower by 314

For the controlled locality trace with 64 frames, LRU faulted less than FIFO at every page size. Larger pages reduced faults because each frame covered more of the fixed byte working set; this does not imply that larger pages are universally better because fragmentation and transfer cost also increase.

5. Requirement 3 - extent-aware MFS

Implementation

Stock MFS allocates data zones through a bitmap. The change first scans for a free run matching mfs_extent_size. If found, its start becomes the exact origin passed to alloc_bit; otherwise the original behavior is retained and a fallback is counted. Only alloc_bit marks a zone allocated, so existing accounting and the on-disk format remain intact.

Requirement 3 — MFS extent preference
minix/fs/mfs/cache.c

```
101   /* If z is 0, skip initial part of the map known to be fully in use. */
102   if (z == sp->s_firstdatazone) {
103     bit = sp->s_zsearch;
104   } else {
105     bit = (bit_t) (z - (sp->s_firstdatazone - 1));
106   }
107   if (mfs_extent_size > 1 && z == sp->s_firstdatazone) {
108     bit_t run;
109
110     mfs_extent_searches++;
111     run = find_zone_run(sp, bit, mfs_extent_size);
112     if (run != NO_BIT) {
113       bit = run;
114       mfs_extent_hits++;
115     } else {
116       mfs_extent_fallbacks++;
117     }
118   }
119   b = alloc_bit(sp, ZMAP, bit);
120   if (b == NO_BIT) {
121     err_code = ENOSPC;
122     if (print_oos_msg)
123         printf("No space on device %d/%d\n", major(sp->s_dev),
124             minor(sp->s_dev));
125     print_oos_msg = 0;  /* Don't repeat message */
126     return(NO_ZONE);
127   }
128   print_oos_msg = 1;
129   if (z == sp->s_firstdatazone) sp->s_zsearch = b;  /* for next time */
130   return( (zone_t) (sp->s_firstdatazone - 1) + (zone_t) b);
```

Native result

Requested zones	Searches	Preferred-run hits	Fallbacks
1	0	0	0
2	6	6	0
4	6	6	0
8	6	6	0
16	6	6	0
32	6	6	0

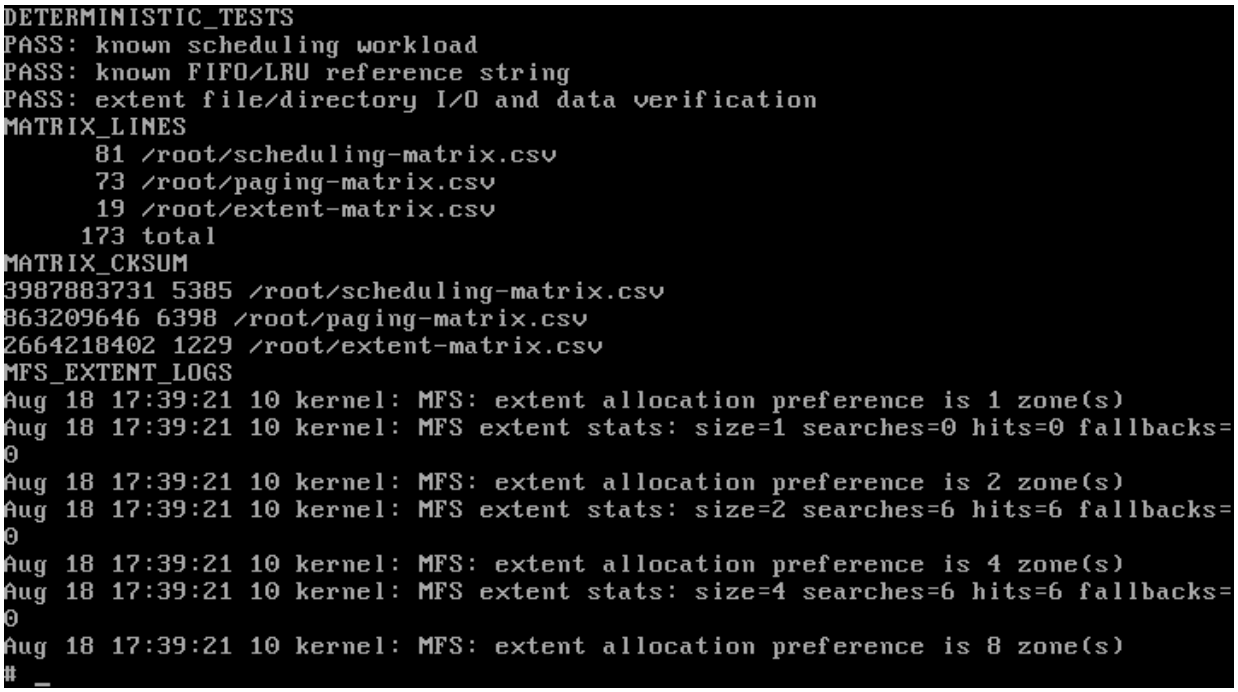
The matrix used a disposable 64 MiB MFS image and produced 18 rows. Every file read matched its deterministic written pattern, so verify_errors stayed zero. All requested runs from 2 through 32 zones were found without fallback. Some short timings rounded to zero because of MINIX 3.3 timer resolution, so correctness and allocator counters are stronger evidence than tiny throughput differences.

6. Native MINIX build and test evidence

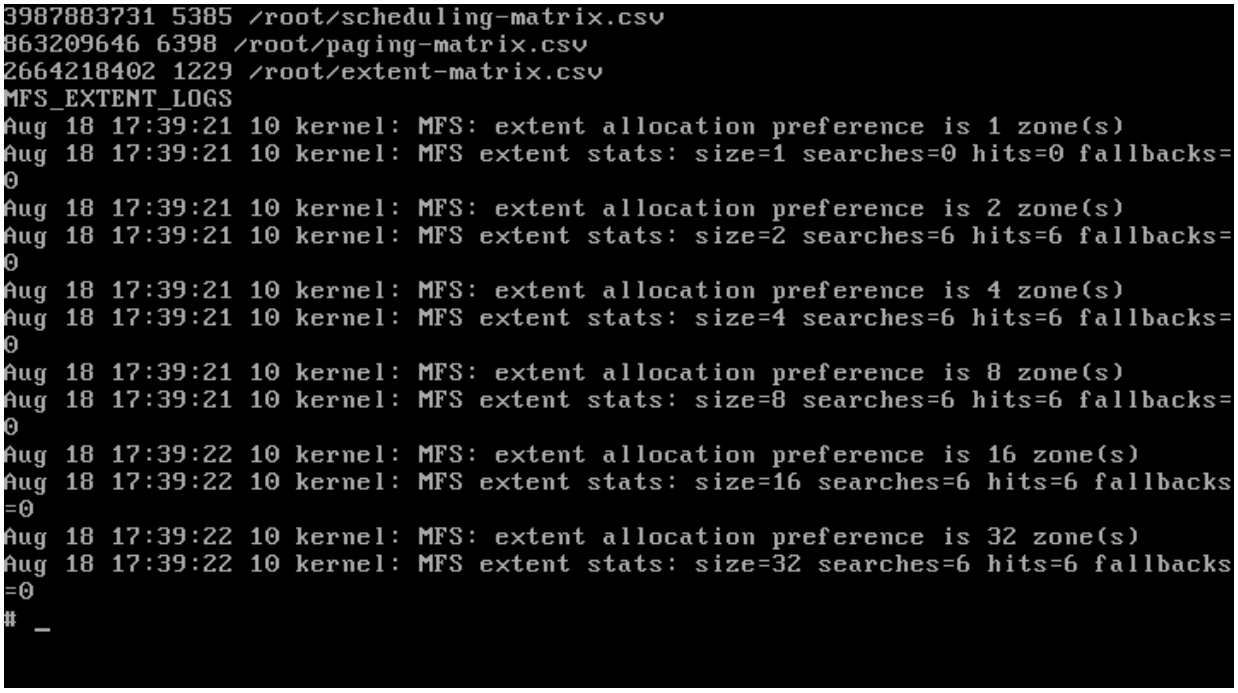
Commands executed in the VM

```
cd /usr/src && make includes
cd /usr/src/minix/commands/schedexperiment && make && make install
cd /usr/src/minix/commands/vmexperiment && make && make install
cd /usr/src/minix/commands/extentexperiment && make && make install
cd /usr/src/minix/fs/mfs && make && make install
cd /usr/src/minix/commands/schedexperiment/tests && sh test_known.sh
cd /usr/src/minix/commands/vmexperiment/tests && sh test_known.sh
cd /usr/src/minix/commands/extentexperiment/tests && sh test_known.sh
run_schedexperiments.sh /root/scheduling-matrix.csv
run_vmexperiments.sh /root/paging-matrix.csv
run_extent_matrix.sh /root/extent-matrix.csv
```

The known tests are deterministic oracles: expected scheduling averages are checked exactly, FIFO/LRU totals are compared with a known reference string, and extent I/O is read back and byte-verified. Matrix collection ran only after all three tests passed.



MFS extent console output



7. Changed files

Area	Main edited files	Purpose
Scheduling	minix/commands/schedexperiment/*	Parser, dispatcher, metrics, test, and matrix script
Paging	minix/commands/vmexperiment/*	Hierarchy, traces, FIFO/LRU, test, and matrix script
MFS	minix/fs/mfs/cache.c, super.c, main.c, mount.c	Exact bitmap origin, free-run scan, option, counters
Extent test	minix/commands/extentexperiment/*	Verified directory/file I/O and CSV metrics
Integration	minix/commands/Makefile; etc/Makefile; etc/*.conf	Build/install entries and configurations

8. Conclusion

All three requirements were built and executed in MINIX 3.3.0. The native correctness gate passed for scheduling, FIFO/LRU paging, and extent file/directory I/O. The matrices contain 80 scheduling rows, 72 paging rows, and 18 extent rows; their checksums are preserved. Requirement 1 demonstrates responsiveness versus dispatch cost, Requirement 2 shows lower LRU faults for the chosen locality workload, and Requirement 3 confirms successful contiguous-run preference with safe fallback and correct data.

Limitations are explicit: scheduler dispatches are experiment events rather than kernel context-switch counters; paging faults are simulated rather than hardware telemetry; and coarse MINIX timing weakens very short throughput measurements. These do not invalidate deterministic correctness or configuration-controlled comparisons. Archived matrices remain tied to configuration, test oracles, and checksums for independent reproducibility.

Submission evidence

- Raw results: project-deliverables/native-results/*.csv and native-validation.txt
- Source comparison: git diff baseline-v3.3.0..HEAD

9. Research basis and MINIX internal structure

MINIX 3 follows a microkernel architecture in which the kernel retains mechanisms that require privileged execution, while many services traditionally placed in a monolithic kernel execute as isolated user-mode processes. The process manager, virtual file-system service, memory-related services, device drivers, and concrete file-system servers communicate through explicit interprocess communication. This organization reduces the trusted kernel surface and provides fault isolation, but it also means that a modification must be placed at the correct architectural boundary. The project therefore treats the scheduler and paging requirements as controlled native experiments and places the persistent allocation-policy change in the MFS server. This interpretation is consistent with the MINIX 3 design literature and source organization [1, 2].

CPU scheduling is evaluated with turnaround time, waiting time, response time, makespan, and dispatch count. Turnaround is completion minus arrival; waiting is turnaround minus demanded CPU service; response is first start minus arrival. Round Robin emphasizes fairness through bounded quanta. Shortest Job First minimizes average waiting for a known batch under its ideal assumptions, but it can delay long jobs. Static priority scheduling expresses importance directly but can starve low-priority work without aging. A multilevel feedback queue adapts priority from observed behavior, favoring short or interactive jobs while moving CPU-bound jobs toward longer quanta. These definitions and trade-offs follow standard operating-systems treatments [3, 4].

Hierarchical paging divides a virtual address into a page offset and one index per configured level. Only leaves reached by the trace need to exist in the sparse experimental structure. FIFO replaces the page that has resided in memory longest, whereas LRU replaces the page whose latest reference is oldest. FIFO requires little history but can exhibit Belady's anomaly; stack algorithms such as LRU do not have that anomaly under the ideal reference model. The experiment reports faults, replacements, hits, remaining empty frames, hierarchy nodes, and lookup work so that replacement behavior and translation-structure cost are not confused [3, 5].

MINIX File System uses allocation bitmaps to record free and used zones. A file's inode identifies data through direct and indirect zone references, while directories are files containing name-to-inode mappings. The extent preference implemented here does not replace this format. It searches the existing zone bitmap for a user-requested contiguous free run and selects the beginning of that run as the next allocation origin. The normal bitmap allocator still performs the actual state change, which preserves existing accounting and compatibility. The design is intentionally a preference layer rather than a new persistent extent tree [1, 2].

Experimental validity and interpretation

The scheduling comparison holds arrival time, demanded service, priority, worker computation, and output definitions constant while changing only the selection policy. Logical time prevents host or VM load from changing the turnaround and waiting calculations, while measured wall time confirms that the child processes performed real work. The 80-row matrix covers four policies, four configured quanta, and five processes. Its most important internal checks are that completion never precedes arrival, waiting equals turnaround minus burst, and every policy completes the same total demanded service. These controls support comparisons within the selected workload; they do not establish that one policy is best for every workload or that the experiment's dispatch count equals a kernel context-switch counter.

The paging matrix changes page size, hierarchy geometry, frame count, and replacement policy through explicit configuration. A fixed trace seed gives FIFO and LRU the same address sequence, so differences are attributable to replacement decisions rather than random input. Each row is checked against the identities references = hits + faults and replacements <= faults. The parser also rejects a geometry unless the level-index widths plus the page offset equal the configured address width. Consequently, hierarchy memory cost and replacement behavior can be compared without confusing them. The reported faults remain model events generated by the native experiment, not hardware faults raised by the processor for an arbitrary MINIX process.

The extent experiment separates correctness from timing. A disposable 64 MiB MFS image provides a controlled free-space target, and the mount option and benchmark configuration use the same requested extent size for each case.

Every iteration writes a deterministic pattern, flushes it, reads the complete file, compares every byte, removes the file, and records allocator counters. Across 18 rows, verification errors and allocation fallbacks were zero for the tested image state. This establishes correct I/O and successful preferred-run selection for those cases. It does not establish an atomic persistent extent format, because normal MFS zone references and bitmap updates remain in use. Several operations also rounded to zero microseconds under the MINIX timer, so allocator hits and byte verification provide stronger evidence than small throughput differences. Larger files, repeated fresh images, and median timing would be appropriate extensions for a stronger performance study.

10. Detailed Requirement 1 implementation and analysis

Requirement 1 - CPU scheduling

What the specification asks for

Implement Round Robin, Shortest Job First, priority-based scheduling, and a Multi-Level Feedback Queue in MINIX. Parameters must be editable in a configuration file. Real processes must execute, and average turnaround and waiting times must be measured and compared.

What changed from stock MINIX

Stock MINIX already schedules processes. The project adds a new native command, configuration file, deterministic test, and matrix runner for controlled policy comparison. It does not delete or replace stock scheduling code. The added dispatcher uses real children but owns a logical policy clock, allowing all four algorithms to see the same arrivals and bursts.

End-to-end execution flow

```
/etc/scheduler.conf
        |
        v
config.c parses algorithms, quanta, work scale, and process rows
        |
        v
schedexperiment.c starts one independent schedule_run() per policy
        |
        v
scheduler.c forks workers and chooses the next ready job
        |
        v
parent sends run_ms -> child performs CPU work -> child acknowledges
        |
        v
parent updates logical time, remaining burst, queue, and completion
        |
        v
metrics are calculated and written to CSV
```

Running policies independently is important: each algorithm begins from the same original workload rather than inheriting completion state from the policy before it.

Implementation summary

schedexperiment runs inside MINIX and creates one real child process for every configured job. The parent implements the four policy selectors and controls which child executes a logical CPU slice through pipes. Each child performs real deterministic CPU work and replies when its slice finishes. Logical time makes the policy metrics reproducible, while wall time proves that actual work ran. Results are written per process to CSV.

Architecture and code map

File	Responsibility
etc/scheduler.conf	Algorithm, quanta, work scale, output, and workload
minix/commands/schedexperiment/config.c	Parses and validates configuration
minix/commands/schedexperiment/scheduler.c	Creates workers, selects jobs, dispatches slices, calculates results
minix/commands/schedexperiment/schedexperiment.c	Runs requested policies and writes CSV
minix/commands/schedexperiment/tests/	Known workload and exact-answer test
minix/commands/schedexperiment/run_schedexperiments.sh	Quantum experiment matrix

Important functions in scheduler.c:

- create_children() creates the real worker processes and pipes.
- select_rr() uses a rotating cursor among ready jobs.
- select_sjf() chooses the ready job with the smallest remaining burst.
- select_priority() chooses the smallest numeric priority.
- select_mlfq() chooses the oldest ready job in the highest available queue.
- schedule_run() is the main dispatch loop and metric collector.

Key implementation excerpts

The following source excerpts identify the central implementation points. Line numbers refer to the submitted commit.

1. Real process creation — scheduler.c:134-182

```
for (index = 0; index < cfg->job_count; index++) {
     int command_pipe[2];
     int reply_pipe[2];
     pid_t pid;

     states[index].cfg = &cfg->jobs[index];
     states[index].remaining_ms = cfg->jobs[index].burst_ms;
     states[index].command_fd = -1;
     states[index].reply_fd = -1;
     states[index].queue_stamp = index;
     if (pipe(command_pipe) != 0) {
             snprintf(error, error_size, "pipe failed: %s", strerror(errno));
             close_children(states, index, 1);
             return -1;
     }
     if (pipe(reply_pipe) != 0) {
             snprintf(error, error_size, "pipe failed: %s", strerror(errno));
             close(command_pipe[0]);
             close(command_pipe[1]);
             close_children(states, index, 1);
             return -1;
     }
     pid = fork();
     if (pid < 0) {
             snprintf(error, error_size, "fork failed: %s", strerror(errno));
             close(command_pipe[0]);
             close(command_pipe[1]);
             close(reply_pipe[0]);
             close(reply_pipe[1]);
             close_children(states, index, 1);
             return -1;
     }
     if (pid == 0) {
```

```
                unsigned int prior;

                close(command_pipe[1]);
                close(reply_pipe[0]);
                for (prior = 0; prior < index; prior++) {
                        close(states[prior].command_fd);
                        close(states[prior].reply_fd);
                }
                child_main(command_pipe[0], reply_pipe[1],
                    cfg->jobs[index].burst_ms, cfg->work_scale);
        }
        close(command_pipe[0]);
        close(reply_pipe[1]);
        states[index].pid = pid;
        states[index].command_fd = command_pipe[1];
        states[index].reply_fd = reply_pipe[0];
}
```

What it proves: there is one real fork() per configured process and two pipes for parent-to-child commands and child-to-parent acknowledgements.

Closing unused pipe ends prevents descriptor leaks and ensures correct EOF and blocking behavior. An extra open writer could prevent a reader from ever observing EOF.

2. The four selectors — scheduler.c:211-275

```
/* RR: rotate the cursor after selecting a ready process. */
for (offset = 0; offset < count; offset++) {
        index = (*cursor + offset) % count;
        if (ready(&states[index], now)) {
                *cursor = (index + 1) % count;
                return (int)index;
        }
}

/* SJF: minimum remaining burst, then earlier arrival. */
if (ready(&states[index], now) && (selected < 0 ||
    states[index].remaining_ms < states[selected].remaining_ms ||
    (states[index].remaining_ms == states[selected].remaining_ms &&
    states[index].cfg->arrival_ms < states[selected].cfg->arrival_ms)))
     selected = (int)index;

/* Priority: smaller numeric priority wins, then earlier arrival. */
if (ready(&states[index], now) && (selected < 0 ||
    states[index].cfg->priority < states[selected].cfg->priority ||
    (states[index].cfg->priority == states[selected].cfg->priority &&
    states[index].cfg->arrival_ms < states[selected].cfg->arrival_ms)))
     selected = (int)index;

/* MLFQ: highest queue first, then oldest queue stamp. */
if (ready(&states[index], now) && (selected < 0 ||
    states[index].queue_level < states[selected].queue_level ||
    (states[index].queue_level == states[selected].queue_level &&
    states[index].queue_stamp < states[selected].queue_stamp)))
     selected = (int)index;
```

These are exact decision expressions from the four selector functions. The repeated ready() condition prevents a process from running before arrival.

Tie resolution is deterministic: SJF and priority use arrival time, MLFQ uses a monotonically increasing queue stamp, and RR uses the rotating cursor.

3. Dispatch, slice selection, and MLFQ demotion — scheduler.c:320-383

```
if (algorithm == SCHED_RR)
    selected = select_rr(states, cfg->job_count, now, &cursor);
else if (algorithm == SCHED_SJF)
    selected = select_sjf(states, cfg->job_count, now);
else if (algorithm == SCHED_PRIORITY)
    selected = select_priority(states, cfg->job_count, now);
else
    selected = select_mlfq(states, cfg->job_count, now);

if (algorithm == SCHED_SJF || algorithm == SCHED_PRIORITY)
    slice = states[selected].remaining_ms;
else if (algorithm == SCHED_RR)
    slice = cfg->quantum_ms;
else
    slice = cfg->mlfq_quantum[states[selected].queue_level];
if (slice > states[selected].remaining_ms)
    slice = states[selected].remaining_ms;

command.run_ms = slice;
if (write_full(states[selected].command_fd, &command,
    sizeof(command)) != 0 ||
    read_full(states[selected].reply_fd, &reply, sizeof(reply)) != 0 ||
    reply.completed_ms != slice) {
    snprintf(error, error_size, "worker %s failed",
        states[selected].cfg->name);
    close_children(states, cfg->job_count, 1);
    return -1;
}

now += slice;
states[selected].remaining_ms -= slice;
result->context_switches++;

/* Executed when an MLFQ job still has remaining work. */
if (states[selected].queue_level + 1 < SCHED_MLFQ_LEVELS)
    states[selected].queue_level++;
states[selected].queue_stamp = stamp++;
```

SJF and priority run the entire remaining burst because their implemented forms are non-preemptive. RR and MLFQ instead use configured quanta.

4. Metric calculation — scheduler.c:392-408

```
job = &result->jobs[index];
strcpy(job->name, cfg->jobs[index].name);
job->arrival_ms = cfg->jobs[index].arrival_ms;
job->burst_ms = cfg->jobs[index].burst_ms;
job->priority = cfg->jobs[index].priority;
job->start_ms = states[index].start_ms;
job->completion_ms = states[index].completion_ms;
job->turnaround_ms = job->completion_ms - job->arrival_ms;
job->waiting_ms = job->turnaround_ms - job->burst_ms;
job->response_ms = job->start_ms - job->arrival_ms;
result->average_turnaround_ms += job->turnaround_ms;
result->average_waiting_ms += job->waiting_ms;
result->average_response_ms += job->response_ms;

result->average_turnaround_ms /= cfg->job_count;
result->average_waiting_ms /= cfg->job_count;
```

```
result->average_response_ms /= cfg->job_count;
```

Waiting time equals turnaround minus burst because turnaround includes both execution and non-execution time. Subtracting total CPU service leaves the logical time spent waiting.

Configuration parameters

```
algorithm=ALL
quantum_ms=20
mlfq_quanta_ms=10,20,40
work_scale=20000
csv_output=/tmp/schedexperiment.csv
process=P1,0,80,3
```

Each process row is name, arrival time, burst time, priority. A smaller numeric priority means higher priority. work_scale controls how much integer work represents one logical burst millisecond; it does not change logical scheduling results.

How each algorithm works here

Round Robin

RR selects ready processes in cyclic order and runs each for at most quantum_ms. If the remaining burst is smaller than the quantum, it runs only the remainder. A small quantum improves response and fairness but causes more dispatches. A very large quantum approaches FCFS behavior.

Shortest Job First

The implementation is non-preemptive. From the jobs that have already arrived, it selects the smallest remaining burst and runs it to completion. SJF often minimizes mean waiting time when burst lengths are known, but long jobs can wait and real systems usually do not know future CPU bursts exactly.

Static priority

The implementation is non-preemptive. It selects the ready process with the smallest numeric priority and runs it to completion. It expresses importance, but low-priority jobs can starve because this experiment does not implement aging.

MLFQ

New jobs begin in queue 0. Queue 0 has the smallest quantum, and lower queues have larger quanta. An unfinished process is demoted after its slice. This gives short or interactive-looking jobs quick service without requiring a supplied burst estimate. The implementation has three queues and no periodic priority boost, which is an explicit limitation of the experiment.

Metrics and formulas

For each process:

```
turnaround = completion - arrival
waiting    = turnaround - burst
response   = first_start - arrival
```

The command also records average values, logical makespan, dispatch count, and measured wall elapsed time. The CSV calls dispatches a context-switch proxy; they are not exact kernel context-switch counters.

Native known-workload result

Algorithm	Avg turnaround	Avg waiting	Avg response	Makespan	Dispatches
RR	185 ms	133 ms	17 ms	260 ms	14
SJF	137 ms	85 ms	85 ms	260 ms	5
Priority	155 ms	103 ms	103 ms	260 ms	5
MLFQ	191 ms	139 ms	5 ms	260 ms	15

Interpretation: SJF achieved the lowest average waiting and turnaround on this workload. MLFQ gave the fastest initial response but needed the most dispatches. The makespan is always 260 ms because all policies eventually execute the same total burst and there is no final idle gap.

How these results were produced

The known configuration defines five jobs with total burst 80 + 40 + 60 + 30 + 50 = 260 ms. RR uses a 20 ms quantum. MLFQ uses 10, 20, and 40 ms queues. test_known.sh runs the command, reads the resulting CSV with awk, and rejects the run unless each algorithm has the expected average turnaround and waiting values.

The full matrix is produced by run_schedexperiments.sh:

1. Loop through base quanta 5, 10, 20, and 40 ms.
2. Generate a temporary configuration with the same five jobs.
3. Set MLFQ quanta to q, 2q, and 4q.
4. Run all four policies.
5. Prefix every CSV row with the current base quantum.
6. Append the rows into one matrix.

There are 4 quanta × 4 algorithms × 5 processes = 80 data rows. SJF and priority do not use the quantum, so their logical results act as controls and should remain unchanged. RR and MLFQ dispatch behavior changes with quantum.

One result traced by hand

For any job, take arrival_ms, start_ms, completion_ms, and burst_ms from its CSV row. Example calculation:

```
turnaround = completion_ms - arrival_ms
waiting    = turnaround - burst_ms
response   = start_ms - arrival_ms
```

The reported algorithm average is the sum of the five per-job values divided by five. This is exactly what scheduler.c:392-408 implements.

Build and execution procedure

```
cd /usr/src/minix/commands/schedexperiment
make clean && make && make install
cp /usr/src/etc/scheduler.conf /etc/scheduler.conf

cd tests && sh test_known.sh
run_schedexperiments.sh /root/scheduling-matrix.csv
wc -l /root/scheduling-matrix.csv
```

The final wc should report 81 lines: one header plus 80 data rows.

Validation considerations

- schedexperiment: not found: run make install or use the built binary path.
- Configuration error: check that every process has name, arrival, burst, and

priority and that all quanta are positive.

- Different wall time: expected under VM load; logical results should remain

identical.

- Averages fail: inspect the first failing algorithm in the known-test output

before running the full matrix.

Design rationale and limitations

Use of real processes

Yes. The parent calls fork() once per job. Children block on command pipes, perform CPU work when dispatched, and acknowledge completion. The policy clock is logical for reproducibility, but the workers are real MINIX processes.

Relationship to the production MINIX scheduler

No. This is a controlled dispatcher compiled and executed inside MINIX. It satisfies real-process execution and policy comparison, but it is not a kernel scheduler replacement.

Use of logical time

VM load and host scheduling make wall time noisy. Logical time makes turnaround, waiting, and response exactly repeatable for a defined workload. Wall time is still recorded to confirm actual work executed.

SJF preemption model

No. It is non-preemptive SJF. The preemptive form would be Shortest Remaining Time First and would re-evaluate when a new process arrives.

Tie-breaking rules

Deterministically, using arrival/order information rather than arbitrary array behavior. This ensures identical configuration produces identical logical results.

MLFQ waiting time compared with SJF

MLFQ optimizes responsiveness without knowing future bursts. It repeatedly serves and demotes jobs, which improves first response but may delay completion and increase dispatch overhead. SJF has the unfair advantage of known bursts.

Handling idle intervals

The logical clock jumps to the next configured arrival instead of dispatching a future process or busy-waiting.

Scope limitations

- Dispatch count is an experiment-level metric, not an exact kernel context-switch count.
- The production MINIX scheduler is not replaced by this experiment.
- The implemented SJF and priority policies are non-preemptive.
- Algorithm comparisons are specific to the configured workload.

11. Detailed Requirement 2 implementation and analysis

Requirement 2 - hierarchical paging and replacement

What the specification asks for

Implement configurable hierarchical paging with user-defined page size, levels, and address format. Implement FIFO and LRU replacement, collect page faults, empty frames, and related metrics while page size and hierarchy depth change, and analyze the results.

What changed from stock MINIX

Stock 32-bit MINIX uses architecture-defined paging and cannot accept arbitrary hardware page geometry from a text file. The project therefore adds a native command that constructs the requested hierarchy in software and compares FIFO and LRU over controlled traces. This is separate from the production VM server and preserves the real system's safety.

End-to-end execution flow

```
/etc/paging.conf
```

```
        |
        v
config.c validates page size, levels, index bits, frames, and trace
        |
        v
simulator.c generates one deterministic byte-address trace
        |
        v
page_lookup() walks/creates sparse hierarchy paths
        |
        v
access_page() records hit or fault and select_frame() applies FIFO/LRU
        |
        v
vmexperiment.c writes policy and hierarchy metrics to CSV
```

FIFO and LRU receive the same address trace. Without this control, a difference in fault counts could come from different input rather than replacement policy.

Implementation summary

vmexperiment is compiled and run natively in MINIX. It validates a configurable virtual-address split, constructs a sparse multi-level page-table structure, generates a deterministic byte-address trace, and sends the same trace through FIFO and exact LRU frame replacement. It records hits, faults, replacements, empty frames, and hierarchy memory in CSV. It is an OS experiment model; it does not claim to reprogram the fixed x86 MMU.

Architecture and code map

File	Responsibility
etc/paging.conf	Address geometry, frames, algorithms, trace, output
minix/commands/vmexperiment/config.c	Parsing and geometry validation
minix/commands/vmexperiment/simulator.c	Sparse hierarchy, trace generation, FIFO/LRU, metrics
minix/commands/vmexperiment/vmexperiment.c	Runs both policies and writes CSV
minix/commands/vmexperiment/tests/	Known reference-string oracle
minix/commands/vmexperiment/run_vmexperiments.sh	72-row controlled matrix

Important functions:

- page_lookup() extracts level indices and creates sparse paths.
- select_frame() chooses an empty frame or a FIFO/LRU victim.
- access_page() counts references, hits, faults, and replacements.
- Trace generators create sequential, random, locality, or file-based input.

Key implementation excerpts

1. Configuration geometry gate — config.c:271-278

```
if (hierarchy_bits + offset_bits != cfg->address_bits) {
    snprintf(error, error_size,
        "sum(level_bits) + log2(page_size) must equal address_bits");
    return -1;
}
if (cfg->frames == 0) {
    snprintf(error, error_size, "frames must be greater than zero");
    return -1;
}
```

The geometry sum must be exact because every address bit belongs to one level index or to the page offset. Missing or overlapping bits would make translation ambiguous.

2. Extracting a hierarchy index — simulator.c:212-223

```
static uint64_t level_index(const struct vmexp_config *cfg,
     uint64_t virtual_page, unsigned int wanted_level)
{
     unsigned int level;
     unsigned int lower_bits;
     uint64_t mask;

     lower_bits = 0;
     for (level = wanted_level + 1; level < cfg->levels; level++)
             lower_bits += cfg->level_bits[level];
     mask = ((uint64_t)1 << cfg->level_bits[wanted_level]) - 1;
     return (virtual_page >> lower_bits) & mask;
}
```

The shift discards indices below the requested level; the mask keeps only that level's configured width.

The hierarchy receives a virtual page rather than a byte address because division by page size removes the offset bits before index extraction.

3. Sparse hierarchy creation — simulator.c:236-262

```
node = sim->root;
for (level = 0; level < sim->cfg->levels; level++) {
     index = level_index(sim->cfg, virtual_page, level);
     slot = slot_find(node, index);
     if (slot == NULL) {
             if (!create)
                     return NULL;
             slot = slot_create(sim, node, index);
             if (slot == NULL)
                     return NULL;
             if (level + 1 == sim->cfg->levels) {
                     page = calloc(1, sizeof(*page));
                     if (page == NULL)
                             return NULL;
                     page->virtual_page = virtual_page;
                     slot->value = page;
                     sim->result.page_table_bytes += sizeof(*page);
             } else {
                     child = node_create(sim, level + 1);
                     if (child == NULL)
                             return NULL;
                     slot->value = child;
             }
     }
     if (level + 1 == sim->cfg->levels)
             return (struct page_entry *)slot->value;
     node = (struct table_node *)slot->value;
}
```

The hierarchy is sparse because slot_create() and node_create() run only when a referenced index is missing; untouched address regions consume no lower-level nodes.

4. FIFO/LRU victim selection — simulator.c:274-292

```
for (frame = 0; frame < sim->cfg->frames; frame++) {
     if (sim->frames[frame] == NULL)
             return frame;
}
selected = 0;
selected_time = policy == POLICY_FIFO ?
    sim->frames[0]->loaded_at : sim->frames[0]->last_access;
```

```
for (frame = 1; frame < sim->cfg->frames; frame++) {
    uint64_t candidate_time;

    candidate_time = policy == POLICY_FIFO ?
        sim->frames[frame]->loaded_at :
        sim->frames[frame]->last_access;
    if (candidate_time < selected_time) {
            selected = frame;
            selected_time = candidate_time;
    }
}
return selected;
```

Victim selection differs only in the timestamp comparison: FIFO uses loaded_at, whereas LRU uses last_access. Both policies use empty frames before evicting a resident page.

5. Hit, fault, and replacement accounting — simulator.c:303-325

```
virtual_page = address / sim->cfg->page_size;
page = page_lookup(sim, virtual_page, 1);
if (page == NULL)
    return -1;
sim->tick++;
sim->result.references++;
if (page->present) {
    sim->result.hits++;
    page->last_access = sim->tick;
    return 0;
}
sim->result.page_faults++;
frame = select_frame(sim, policy);
victim = sim->frames[frame];
if (victim != NULL) {
    victim->present = 0;
    sim->result.replacements++;
}
page->present = 1;
page->frame = frame;
page->loaded_at = sim->tick;
page->last_access = sim->tick;
sim->frames[frame] = page;
```

Every replacement is caused by a fault, but an initial load into an empty frame faults without replacing a resident page.

Address geometry

If address width is A, page size is 2^p, and hierarchy index widths are b1 ... bn, a configuration is valid only when:

```
b1 + b2 + ... + bn + p = A
```

For the default 32-bit address and 4 KiB page, the offset is 12 bits. Two 10-bit levels consume the remaining 20 bits. The virtual page number is address / page_size; masks and shifts extract indices from it.

The hierarchy is sparse: it allocates only paths touched by the trace. This is why the experiment can compare table nodes, entries, and approximate memory instead of allocating every theoretical entry.

Configuration parameters

```
address_bits=32
page_size=4096
levels=2
level_bits=10,10
frames=64
algorithm=BOTH
trace_mode=locality
```

```
references=10000
working_set_bytes=1048576
hot_bytes=131072
access_stride=64
seed=335
```

The trace uses byte addresses, not page numbers. Keeping the byte working set fixed is essential when page size changes; otherwise increasing page size would also silently increase the represented workload.

FIFO and LRU

FIFO stores loaded_at. A hit does not change FIFO order. When frames are full, the page with the oldest load time is evicted. FIFO is simple but ignores locality and may show Belady's anomaly.

LRU stores last_access. A load and every hit refresh recency. When frames are full, the least recently accessed page is evicted. Exact LRU is a stack algorithm, so adding frames cannot increase faults for a fixed trace. Real kernels often approximate LRU because updating exact order on every access is expensive.

Metrics and invariants

- references: all address accesses.
- hits: requested page already resident.
- page_faults: requested page not resident, including first fills.
- replacements: faults that evicted a page after frames became full.
- empty_frames: unused frames after the trace.
- hit_ratio: hits divided by references.
- hierarchy nodes, entries, and bytes: sparse metadata cost.

The key invariant is:

```
references = hits + page_faults
```

First fills are faults but not replacements. Therefore replacements cannot be greater than faults.

Native results

For the controlled locality trace with 64 frames:

Page size	FIFO faults	LRU faults
1,024 B	6,937	6,791
2,048 B	4,735	4,047
4,096 B	2,666	1,655
8,192 B	1,378	1,064

LRU performed better on this locality-heavy workload because hot pages refresh their recency. Faults decreased as page size increased because every frame covered more of the same fixed byte working set. Larger pages are not always better: they may increase internal fragmentation and transfer unused data.

Changing levels primarily changes hierarchy depth and metadata, not policy faults, when the page size, frames, and trace are identical.

How these results were produced

run_vmexperiments.sh fixes a 10,000-reference locality trace, 1 MiB byte working set, 128 KiB hot region, 64-byte stride, and seed 335. It varies:

- frames: 16, 32, and 64;
- page size: 1,024, 2,048, 4,096, and 8,192 bytes;
- hierarchy depth: one, two, and three levels; and
- replacement policy: FIFO and LRU.

For every page size, the script generates valid level-bit splits. For example, 4 KiB pages have a 12-bit offset and 20 hierarchy bits, represented as 20, 10,10, or 7,7,6. The matrix therefore contains 3 × 4 × 3 × 2 = 72 data rows.

The result table selects the two-level, 64-frame rows. Faults fall with larger pages because the byte workload is fixed; each resident page covers more bytes. LRU beats FIFO on these locality rows because hot-page hits refresh last_access, while FIFO never changes loaded_at on a hit.

Why hierarchy depth did not change those fault counts

All valid level splits map the same byte address to the same virtual page. They change how the lookup path is divided and how much sparse metadata is allocated, but the final page identity and replacement trace remain the same.

Build and execution procedure

```
cd /usr/src/minix/commands/vmexperiment
make clean && make && make install
cp /usr/src/etc/paging.conf /etc/paging.conf

cd tests && sh test_known.sh
run_vmexperiments.sh /root/paging-matrix.csv
wc -l /root/paging-matrix.csv
```

The expected size is 73 lines: one header plus 72 data rows.

The matrix script waits until a complete three-line single-case CSV exists. On this particular MINIX image, libc teardown may spin after the file is closed, so the script terminates only that completed process and then consumes the durable CSV. A case is rejected if the complete file does not appear before timeout.

Validation considerations

- Geometry error: recalculate log2(page_size) and ensure level bits plus the

offset equal address_bits.

- Missing CSV: check /tmp/vmexperiment-matrix.log for the case error.
- Different FIFO/LRU inputs: keep seed and trace parameters identical and use

algorithm=BOTH.

- Unexpected NA fields: real VM telemetry is intentionally not linked on this

image; simulated metrics remain valid and explicitly labelled.

Known-answer test

The trace is 1 2 3 4 1 2 5 1 2 3 4 5 with three frames. Expected faults:

```
FIFO = 9
LRU  = 10
```

LRU is not guaranteed to beat FIFO on every individual reference string. Its advantage appears for workloads with useful temporal locality.

Design rationale and limitations

Relationship to the hardware page table

No. It is a configurable hierarchy and replacement experiment running inside MINIX. The x86 hardware page format is fixed and cannot be changed arbitrarily from a text file. The final CSV leaves unreliable real VM telemetry fields as NA rather than mislabelling simulator results as hardware faults.

Purpose of hierarchical paging

A flat table needs space for all virtual pages. A hierarchy allocates lower levels only for used address regions, trading extra lookups for potentially much lower metadata memory on sparse address spaces.

Hierarchy depth and page-fault frequency

Not by itself. Replacement faults depend on the page-reference sequence, page size, frames, and policy. More levels primarily change translation structure and metadata overhead.

Effect of page size in the collected results

The trace represents a fixed number of bytes. A larger page brings more of that working set into each frame, so fewer distinct pages are needed. The trade-off is possible fragmentation and unnecessary data transfer.

Known-test comparison of LRU and FIFO

Policy quality depends on the trace. The known string happens to favor FIFO. LRU's stack property prevents Belady's anomaly but does not mean it wins against every different algorithm on every trace.

Belady's anomaly

For FIFO, increasing the number of frames can sometimes increase faults. Exact LRU cannot exhibit this for a fixed reference string because its resident sets have the stack property.

Use of a fixed trace seed

It produces identical traces across policies and configurations, so observed differences come from the intended independent variable rather than random input changes.

Scope limitations

- The experiment does not change the CPU's hardware page size.
- Simulator faults are distinct from hardware faults generated by a real process.
- Increasing hierarchy depth does not automatically reduce page faults.
- LRU is not guaranteed to produce fewer faults than FIFO for every trace.

12. Detailed Requirement 3 implementation and analysis

Requirement 3 - MFS extent allocation

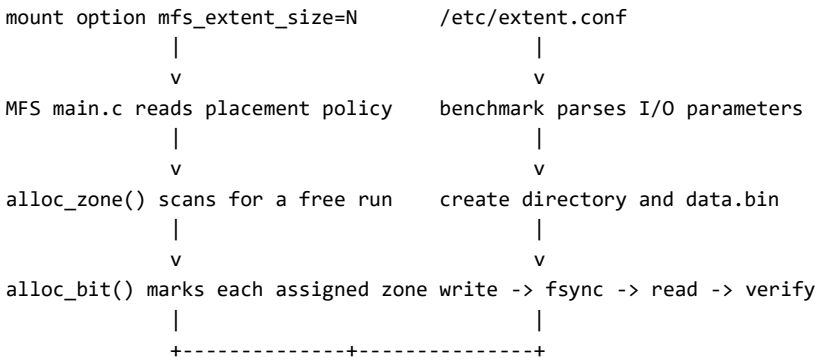
What the specification asks for

Explain how MINIX manages free disk space, modify allocation to use a user-defined extent size, configure it from a file or service option, compare performance as extent blocks change, and understand how MINIX creates, reads, writes, and removes files and directories.

What changed from stock MINIX

Stock MFS begins bitmap allocation near a preferred zone but does not first require a user-selected free-run length. The project adds a read-only run scan, an exact first-bit origin, a bounded service option, counters, and a native I/O benchmark. The inode format, direct/indirect zone references, and freeing logic remain unchanged.

End-to-end execution flow



```
                    v
CSV timings/errors + MFS counters
```

There are two controls because placement belongs to the MFS server, while file size, chunking, repetitions, and output belong to the benchmark process.

Implementation summary

MFS manages free inodes and data zones with bitmaps. The project modifies the real MFS allocation path so the first zone allocation can search for a free run of mfs_extent_size zones and start normal bitmap allocation at the exact run origin. If no run exists, MFS safely falls back to stock allocation. A native benchmark performs real directory creation, file creation, deterministic writes, fsync, reads with byte verification, unlink, and directory removal on a disposable MFS image.

Blocks, zones, inodes, and bitmaps

- A block is a unit of device/file-system I/O.
- An MFS zone is the allocation unit referenced by file inodes. Depending on

file-system geometry, a zone can contain one or more blocks.

- An inode stores file metadata and direct/indirect zone references.
- The inode bitmap tracks free/used inodes.
- The zone bitmap tracks free/used data zones. A zero bit is free; allocation

sets it and freeing clears it.

- A directory is a special file containing name-to-inode mappings.

Architecture and code map

File	Responsibility
minix/fs/mfs/cache.c	alloc_zone(), free-run search, hit/fallback counters
minix/fs/mfs/super.c	alloc_bit() exact-origin bitmap scan
minix/fs/mfs/main.c	Reads mfs_extent_size when the MFS service starts
minix/fs/mfs/glo.h, proto.h	Shared state and function declarations
minix/fs/mfs/mount.c	Reports counters at unmount
etc/extent.conf	Benchmark chunk size, file size, repetitions, output
minix/commands/extentexperiment/	Real directory and verified file-I/O benchmark
run_extent_matrix.sh	Remounts scratch MFS for sizes 1, 2, 4, 8, 16, 32

Key implementation excerpts

1. Finding consecutive free zone bits — cache.c:47-75

```
static bit_t find_zone_run_range(struct super_block *sp, bit_t first,
  bit_t limit, unsigned int length)
{
  bit_t candidate, probe;
  unsigned int offset;

  candidate = first < 1 ? 1 : first;
  while (candidate < limit && (bit_t) length <= limit - candidate) {
    for (offset = 0; offset < length; offset++) {
      probe = candidate + (bit_t) offset;
      if (!zone_bit_is_free(sp, probe)) break;
    }
```

```
    if (offset == length) return(candidate);
    candidate += (bit_t) offset + 1;
  }
  return(NO_BIT);
}

static bit_t find_zone_run(struct super_block *sp, bit_t origin,
  unsigned int length)
{
  bit_t map_bits, found;

  map_bits = (bit_t) (sp->s_zones - (sp->s_firstdatazone - 1));
  if (origin >= map_bits) origin = 1;
  found = find_zone_run_range(sp, origin, map_bits, length);
  if (found == NO_BIT && origin > 1)
    found = find_zone_run_range(sp, 1, origin, length);
  return(found);
}
```

The run scan wraps by searching from the normal origin to the bitmap end and, if unsuccessful, from bit 1 back to the origin.

2. Connecting the run to normal allocation — cache.c:101-119

```
if (z == sp->s_firstdatazone) {
    bit = sp->s_zsearch;
} else {
    bit = (bit_t) (z - (sp->s_firstdatazone - 1));
}
if (mfs_extent_size > 1 && z == sp->s_firstdatazone) {
    bit_t run;

    mfs_extent_searches++;
    run = find_zone_run(sp, bit, mfs_extent_size);
    if (run != NO_BIT) {
      bit = run;
      mfs_extent_hits++;
    } else {
      mfs_extent_fallbacks++;
    }
}
b = alloc_bit(sp, ZMAP, bit);
```

This is the core modification: the new code chooses a better origin, but the existing alloc_bit() remains the only operation that marks a zone allocated.

The fallback remains safe because a NO_BIT result leaves the original search origin unchanged before alloc_bit() is called.

3. Honoring the exact origin bit — super.c:64-108

```
block = (block_t) (origin / FS_BITS_PER_BLOCK(sp->s_block_size));
word = (origin % FS_BITS_PER_BLOCK(sp->s_block_size)) / FS_BITCHUNK_BITS;
first_bit = origin % FS_BITCHUNK_BITS;

for (wptr = &b_bitmap(bp)[word]; wptr < wlim; wptr++) {
    if (*wptr == (bitchunk_t) ~0) {
            first_bit = 0;
            continue;
    }

    k = (bitchunk_t) conv4(sp->s_native, (int) *wptr);
    for (i = first_bit; i < FS_BITCHUNK_BITS &&
```

```
            (k & (1 << i)) != 0; ++i) {}
        first_bit = 0;
        if (i == FS_BITCHUNK_BITS) continue;

        b = ((bit_t) block * FS_BITS_PER_BLOCK(sp->s_block_size))
            + (wptr - &b_bitmap(bp)[0]) * FS_BITCHUNK_BITS
            + i;
        if (b >= map_bits) break;

        k |= 1 << i;
        *wptr = (bitchunk_t) conv4(sp->s_native, (int) k);
        MARKDIRTY(bp);
        put_block(bp, MAP_BLOCK);
        if(map == ZMAP) {
                used_blocks++;
                lmfs_blockschange(sp->s_dev, 1);
        }
        return(b);
}
```

The first_bit offset applies only to the initial bitmap word; every later word is scanned from bit zero.

4. Reading the MFS service option — main.c:35-43

```
env_setargs(argc, argv);
mfs_extent_size = 1;
extent_size = 1;
if (env_parse("mfs_extent_size", "d", 0, &extent_size, 1, 1024) ==
    EP_SET)
     mfs_extent_size = (unsigned int) extent_size;
printf("MFS: extent allocation preference is %u zone(s)\n",
     mfs_extent_size);
```

Without the service option, the default is one zone, so the run search is bypassed and ordinary allocation behavior is preserved.

5. Real read-back verification — extentexperiment.c:176-223

```
verify_errors = 0;
gettimeofday(&read_start, NULL);
for (offset = 0; offset < total_bytes; offset += chunk_size) {
     size_t amount;

     amount = chunk_size;
     if ((unsigned long long)amount > total_bytes - offset)
             amount = (size_t)(total_bytes - offset);
     if (read_full(fd, buffer, amount) != 0) {
             snprintf(error, error_size, "read failed: %s", strerror(errno));
             close(fd);
             unlink(filepath);
             rmdir(subdir);
             free(buffer);
             return -1;
     }
     verify_errors += verify_pattern(buffer, amount, offset, iteration);
}

fprintf(csv, "%.3f,%.3f,%u\n", throughput_mib(total_bytes, write_us),
    throughput_mib(total_bytes, read_us), verify_errors);
if (verify_errors != 0) {
     snprintf(error, error_size, "%u data verification errors",
         verify_errors);
```

```
        return -1;
    }
```

Byte verification is evaluated before throughput because a fast write/read result is invalid when the returned data is corrupted.

What changed in the allocator

Stock alloc_zone() converts a preferred physical zone into a zone-bitmap bit and calls alloc_bit(). The project adds find_zone_run() to scan the bitmap for consecutive free bits. For the first zone of a file:

7. Increment the search counter.
8. Search for a run of the requested length, wrapping safely if necessary.
9. If found, use its first bit as the preferred origin and increment hits.
10. If not found, increment fallbacks and retain the original search origin.
11. Call the existing alloc_bit() to perform the actual allocation.

super.c was adjusted so the first bitmap word starts scanning at the exact bit offset, rather than at bit zero of the containing word. Without this change, MFS could find a run beginning inside a word but allocate an earlier free bit.

Why this is safe

The run scan is read-only. It does not reserve all zones in advance. A zone is marked allocated only when the existing allocator assigns it to the inode. Therefore the change keeps stock accounting, freeing, and the on-disk MFS format. If a requested run is unavailable, the original allocator still works.

The trade-off is that the feature is an extent preference, not a hard persistent extent guarantee. Another allocation could consume part of a discovered run before a file grows, although a quiet, serialized scratch experiment minimizes that risk.

Two configuration layers

The MFS service option controls placement:

```
mount -t mfs -o mfs_extent_size=8 /dev/vnd0 /mnt/extenttest
```

The benchmark file controls I/O:

```
extent_blocks=8
block_size=4096
file_blocks=512
iterations=3
directory=/mnt/extenttest/extentexperiment
csv_output=/tmp/extentexperiment.csv
```

These values must match during a controlled experiment. Changing only extent.conf changes the benchmark's write chunk but does not change MFS allocation policy. A new service option requires unmounting and remounting so a new MFS server instance reads it.

File and directory benchmark path

For every iteration, extentexperiment:

12. Creates its owned subdirectory.
13. Creates and opens data.bin.
14. Writes a deterministic byte pattern in configured chunks.
15. Calls fsync() and records allocation information with fstat().
16. Seeks to the start and reads every byte.
17. Verifies the pattern and records verify_errors.
18. Closes and unlinks the file.
19. Removes its subdirectory.

This exercises directory entries, inode creation, zone allocation, cache writes, persistence, reads, deallocation, and directory removal.

Metrics and native results

The CSV records requested blocks, iteration, block and file sizes, total bytes, logical extent count, allocated 512-byte blocks, operation times, throughput, and verification errors.

Requested zones	Searches	Hits	Fallbacks
1	0	0	0
2	6	6	0
4	6	6	0
8	6	6	0
16	6	6	0
32	6	6	0

The native matrix used a disposable 64 MiB image through /dev/vnd0, produced 18 rows, and reported zero data-verification errors. Every requested run above size 1 was found without fallback.

Some short operations measured zero microseconds because the MINIX 3.3 timer is coarse. Do not interpret those zeros as infinite performance. Correct data and allocator counters are stronger evidence than small throughput differences in this run.

How these results were produced

The native run used a disposable 64 MiB MFS image attached through /dev/vnd0. EXTENT_DEVICE=/dev/vnd0 caused run_extent_matrix.sh to perform this sequence for sizes 1, 2, 4, 8, 16, and 32:

20. Mount the image with mfs_extent_size equal to the current size.
21. Generate a matching benchmark configuration.
22. Write a 2 MiB file (512 × 4096 bytes) three times.
23. Read back and verify every byte.
24. Unlink the file and remove its directory.
25. Unmount so MFS prints search, hit, and fallback counters.
26. Append the three CSV rows to the final matrix.

This gives 6 sizes × 3 repetitions = 18 data rows. Sizes above one show six MFS searches because every iteration performs a first-zone allocation for its new subdirectory and another for its data file. All six searches found a run, so hits are six and fallbacks are zero.

The extent_blocks=1 case is the compatibility baseline. It bypasses the new run search, which is why its counters remain zero.

Interpreting performance responsibly

The timer is too coarse for strong conclusions from very short individual operations. The defensible conclusions are that the real MFS path accepted the requested preferences, the preferred runs were found, all file operations completed, and data verification reported zero errors. A stronger throughput study would enlarge files, repeat runs, reset the image state, and report medians and ranges.

Build and execution procedure

```
cd /usr/src/minix/fs/mfs
make clean && make && make install

cd /usr/src/minix/commands/extentexperiment
make clean && make && make install
cp /usr/src/etc/extent.conf /etc/extent.conf

cd tests && sh test_known.sh
```

```
EXTENT_DEVICE=/dev/vnd0 \
EXTENT_MOUNT_POINT=/mnt/extenttest \
run_extent_matrix.sh /root/extent-matrix.csv

wc -l /root/extent-matrix.csv
```

The expected size is 19 lines: one header plus 18 data rows. Verify /dev/vnd0 before running the matrix; never guess a device name.

Validation considerations

- Mount fails: confirm the image contains MFS and is not already mounted.
- Counters always show size 1: remount with the option and confirm the new MFS

binary was installed.

- CSV writes under /tmp: set the benchmark directory under the scratch mount

when testing real MFS placement.

- Nonzero verify_errors: treat it as a correctness failure and stop analysis.
- Zero-microsecond timings: enlarge the workload; do not divide them into a

claim of infinite throughput.

Design rationale and limitations

Stock MFS free-space allocation

It uses inode and zone bitmaps. alloc_zone() converts a preferred zone to a bitmap origin, and alloc_bit() scans for a zero bit, sets it, marks the bitmap buffer dirty, and updates accounting.

Roles of cache.c and super.c

cache.c finds the start of a sufficiently long free run. super.c must honor that exact bit inside the first bitmap word. Otherwise the allocator may start at an earlier zero bit and defeat the run selection.

Extent reservation behavior

No. Reserving zones without inode references could leak space after a crash and would require new metadata and recovery rules. Existing bitmap allocation marks each zone only when it is assigned.

On-disk format compatibility

No. Inodes still use the normal direct and indirect zone references. The change is an extent-biased placement policy and instrumentation, preserving format compatibility.

Behavior under free-space fragmentation

If no run of the requested size exists, MFS increments the fallback counter and uses the original allocation search. Correctness is preserved, but contiguity is not guaranteed.

Use of a disposable MFS image

Rebuilding and remounting a file-system server or formatting the wrong device can destroy the VM. A secondary image isolates the experiment and can be reset between cases for comparable free-space conditions.

Relationship between searches and CSV rows

Each of the three iterations creates a new subdirectory and a new data file. Both need a first-zone allocation, so MFS performs two preferred-run searches per iteration: six searches in total. Every search hit and none fell back.

Comparison with a persistent extent implementation

It would require persistent metadata describing extent start and length, atomic reservation, freeing and truncation rules, crash recovery, and updates to tools such as fsck. This project deliberately avoids changing the on-disk format.

Scope limitations

- The entire preferred run is not reserved atomically.
- MFS continues to use its existing zone references rather than a persistent extent tree.
- Zero-microsecond rows reflect timer resolution and do not indicate infinite throughput.
- Formatting or mounting is restricted to a verified disposable device.

13. References

[1] A. S. Tanenbaum and A. S. Woodhull, Operating Systems: Design and Implementation, 3rd ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2006.

[2] MINIX 3 Project, MINIX 3.3.0 source tree and official documentation, Vrije Universiteit Amsterdam, release 3.3.0. The submitted source baseline is preserved under tag baseline-v3.3.0.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.

[4] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA: Pearson, 2015.

[5] L. A. Belady, A study of replacement algorithms for a virtual-storage computer, IBM Systems Journal, vol. 5, no. 2, pp. 78-101, 1966.